

Website Pre-Launch Checker

A detailed build plan: architecture, feature breakdown, and step-by-step implementation guidance for a URL-based site readiness report tool.

1. Overview & Goal

The Website Pre-Launch Checker is a tool that takes a single input — a website URL — and returns a structured report on whether that site is technically ready to go live. It checks things a developer or marketer would otherwise have to verify by hand: does the site load, is it secure, is it mobile-friendly, does it have the metadata search engines expect, and are there any obviously broken pieces (dead links, missing image descriptions, missing sitemap or robots file). The value of the tool is compressing what is normally a 20-minute manual QA pass into a single automated report.

At a high level, the system works by having a backend service fetch the target page (and a few related resources like `robots.txt` and `sitemap.xml`), run a set of independent checks against what it fetched, and hand back a single JSON report that the frontend renders as a checklist.

2. High-Level Architecture

The system is a classic three-layer pipeline: a frontend that collects the URL and displays results, a backend that does all the actual work, and the external target website itself, which the backend treats as an untrusted third party it is making requests to.

```
Frontend (form + report UI)
|
v
Backend API (orchestrates checks)
|-- fetches target URL + related resources (robots.txt, sitemap.xml, HTML)
|-- runs individual checker functions against the fetched data
|-- aggregates results into a single JSON report
|
v
Frontend renders the report (pass / warning / fail per item)
```

The reason to centralize everything in the backend rather than doing checks in the browser is twofold. First, browsers enforce CORS, so client-side JavaScript usually cannot fetch an arbitrary third-party website's HTML directly. Second, keeping the logic server-side means you control rate limiting, timeouts, and caching in one place, and you can add new checks without shipping a new frontend build.

3. Phased Delivery Plan

Rather than building all ten checks and every nice-to-have at once, it's more effective to ship in three phases. Each phase is a fully working product on its own — you are not building half-finished features, you are widening scope.

Phase 1 — MVP

A single URL input, all checks run synchronously in one request/response cycle, and a simple pass/fail report. The goal here is purely to prove the pipeline end-to-end: fetch a page, run a handful of checks against it, render the result. Keep the broken-link checker capped at a small number of links so this phase never hangs.

Phase 2 — Async & Scoring

Once the pipeline works, the slow checks (broken links across a large site, full performance audits) get moved to a background job queue so the user isn't stuck watching a spinner for 30 seconds. This phase also introduces an overall score, and persists reports to a database so users can view history instead of losing the result on refresh.

Phase 3 — Polish

PDF export of the report, scheduled re-checks with email/notification alerts, multi-page crawling instead of just the homepage, and optionally user accounts so reports are tied to a project rather than a one-off scan.

4. Low-Level Plan

4.1 Tech Stack

These are suggestions, not requirements — use whatever stack you're already comfortable in. The important properties are: a backend language with good async/concurrent HTTP support (since you'll be firing off many outbound requests per scan), and an HTML parser that lets you query the DOM the way you would with `document.querySelector`.

Layer	Suggestion	Why
Frontend	React + Tailwind	Form input, results dashboard with pass/warning/fail states and expandable detail per check.
Backend	Node.js + Express	Non-blocking I/O suits a tool that spends most of its time waiting on network requests.
HTML parsing	Cheerio	jQuery-like API for querying tags, attributes, and meta values from raw HTML.
HTTP client	Axios / node-fetch	Needs configurable timeouts and redirect handling for hitting arbitrary third-party sites.
Database (optional in MVP)	Postgres / MySQL / SQLite	Stores past reports once you want history or comparisons.
Queue (Phase 2)	BullMQ + Redis	Moves slow checks (broken links, full-site crawl) off the request/response cycle.

4.2 Core Data Flow — Step by Step

This is what happens, in order, for a single scan. Each step is explained in detail because getting the sequencing and error handling right here is what makes the tool feel reliable instead of flaky.

Step 1 — Validate and normalize the URL

Before making any network call, check that the input is a plausible URL. Users will paste things like `example.com` without a protocol, or with trailing slashes, or with stray whitespace. Normalize by trimming, lowercasing the host, and defaulting to `https://` if no protocol is given. Reject obviously invalid input (empty string, no dot in the host, localhost/internal IPs — more on that in the security section) before you spend a network round trip on it.

Step 2 — Fetch the page

Make the actual HTTP GET request to the normalized URL. Capture three things regardless of whether the request "succeeds": the final status code, the response headers, and the response body. Set an explicit timeout (e.g. 10 seconds) — without one, a single unresponsive site will hang your server. Follow redirects by default (most site migrations use them) but record the final URL you landed on, since a redirect chain itself can be worth surfacing in the report.

Step 3 — Run each checker against the fetched data

Once you have the HTML, headers, and status code, each checker is a small, independent function that takes that data and returns a status (pass / warning / fail) plus supporting details. Independence matters: a bug in the broken-link checker shouldn't take down the title-tag checker. Run the checks that only need the already-fetched HTML synchronously (title, meta description, viewport, alt text), and treat checks that need additional network calls (broken links, robots.txt, sitemap.xml, SSL inspection) as separate async operations, ideally run concurrently rather than one after another.

Step 4 — Aggregate into a single report object

Collect every checker's result into one JSON object keyed by check name, plus a top-level summary (overall score, timestamp, URL scanned). This is the contract between backend and frontend — get its shape right early since both API design and UI rendering depend on it (see section 4.4).

Step 5 — Render the report

The frontend takes the JSON and renders each check as a row: an icon for pass/warning/fail, a short label, and an expandable section with the specific details (e.g. which 5 links were broken, or what the actual meta description text was and why its length triggered a warning). Group related checks visually — SEO checks together, security checks together, performance checks together — rather than presenting all ten as a flat list.

4.3 Feature Breakdown: Each Checker Explained

Below is every check from the original checklist, with the mechanism to implement it and the reasoning behind what counts as pass/warning/fail.

SSL Available

How to implement: Attempt the request on `https://`. If it succeeds, use Node's `tls` module (or inspect the certificate via `https.get`'s socket) to pull certificate validity dates and issuer.

Why it matters / grading logic: Pass if HTTPS loads with a currently-valid certificate. Warning if the certificate is valid but expiring within 30 days. Fail if HTTPS doesn't load at all, or the certificate is expired, self-signed, or the hostname doesn't match the certificate.

Website Accessible

How to implement: A plain GET request to the (redirect-followed) URL, recording the final status code and response time.

Why it matters / grading logic: Pass on a 200 status within a reasonable time window. Warning on slow response times (e.g. over 3 seconds) or a 3xx chain that's unusually long. Fail on 4xx/5xx, DNS failure, or a timeout — these mean the site isn't up, full stop.

Mobile Viewport Configured

How to implement: Parse the fetched HTML's `<head>` with Cheerio and look for `<meta name="viewport">`.

Why it matters / grading logic: Pass if present with a sensible content value (contains `width=device-width`). Fail if absent — this is a strong signal the site will render poorly on phones, which matters for both usability and Google's mobile-first indexing.

Page Title Exists

How to implement: Check for a non-empty `<title>` tag and measure its character length.

Why it matters / grading logic: Pass if present and within roughly 50–60 characters (Google's typical display cutoff). Warning if present but too short or too long. Fail if missing or empty — a page with no title is a basic SEO failure.

Meta Description Exists

How to implement: Check for `<meta name="description">` and measure its length.

Why it matters / grading logic: Pass if present and roughly 150–160 characters. Warning if present but poorly sized (too short to be useful, or long enough that search engines will truncate it). Fail if entirely absent.

Broken Links Detected

How to implement: Extract every `<a href>` from the page, deduplicate them, and send a HEAD request to each (falling back to GET if the server doesn't support HEAD), with a concurrency cap of roughly 5–10 simultaneous requests so you don't hammer the target site or your own outbound connection pool.

Why it matters / grading logic: Pass if all links return 2xx/3xx. Warning if a small number are broken. Fail if a significant fraction return 4xx/5xx. This is the slowest and most failure-prone check — it's the first candidate to

move to a background job once you have a queue, and in the MVP it should be capped to the first 40–50 links found.

Images Missing ALT Text

How to implement: Extract every `<img>` tag and check whether the `alt` attribute exists and is non-empty.

Why it matters / grading logic: Pass if all images have alt text. Warning if a small percentage are missing it. Fail if a large share are missing it — this affects both accessibility (screen readers) and image SEO.

Sitemap Available

How to implement: Try fetching `/sitemap.xml` directly, and separately check `robots.txt` for a Sitemap: directive, since some sites host it at a non-default path.

Why it matters / grading logic: Pass if a sitemap is found by either method and returns valid XML. Fail if neither method locates one — search engines rely on this for efficient crawling.

Robots.txt Available

How to implement: Fetch `/robots.txt` and confirm it returns a 200 status with non-empty content.

Why it matters / grading logic: Pass if present and parseable. Warning if present but empty or if it disallows all crawling (which may be intentional pre-launch, so flag rather than fail). Fail if the request 404s.

Basic Performance Indicators

How to implement: Measure time-to-first-byte (TTFB) from the initial fetch, total HTML payload size in KB, and count of blocking `<script>`/`<link>` tags in the head. For a deeper audit without building your own Lighthouse clone, call Google's free PageSpeed Insights API with the target URL.

Why it matters / grading logic: Pass/warning/fail thresholds here are looser and more about flagging obvious problems (very slow TTFB, unusually heavy HTML) than producing a precise performance score — leave the precise scoring to PageSpeed Insights if you integrate it in Phase 2.

4.4 API Design

One endpoint is enough for the MVP: it accepts a URL and returns the full report. Keeping every check's result under a consistent `{status, details}` shape means the frontend can render all ten with one generic component instead of one-off UI per check.

```
POST /api/check
body: { "url": "https://example.com" }

response: {
  url, checkedAt, overallScore,
  checks: {
    ssl: { status: "pass", details: { ... } },
    accessible: { status: "pass", responseTimeMs, statusCode },
    viewport: { status: "fail" },
    title: { status: "pass", value: "...", length: 45 },
    metaDescription: { status: "warning", value: "...", length: 200 },
    brokenLinks: { status: "warning", total: 24, broken: [...] },
    imagesAlt: { status: "fail", total: 12, missingAlt: 5 },
    sitemap: { status: "pass", url: "/sitemap.xml" },
    robotsTxt: { status: "pass" },
    performance: { status: "warning", ttfbMs, pageSizeKb }
  }
}
```

Every check status is one of three values — pass, warning, fail — so the frontend can use one shared badge component. The `details` field is intentionally loose per-check, since a broken-link list and an SSL certificate's expiry date don't share a shape; the frontend just needs to know how to render each check's specific detail block.

4.5 Build Order — and Why This Sequence

Building in this order means you always have something runnable, and you tackle the hardest, most failure-prone piece (broken links) only after the rest of the pipeline is proven to work.

#	Step	Why this order
1	Basic URL fetch + accessibility check	Proves the end-to-end pipeline (input → fetch → report) before adding complexity.
2	HTML parsing checks (title, meta, viewport, alt text)	All four reuse the single HTML document already fetched in step 1 — no new network calls.
3	robots.txt / sitemap checks	Simple additional fetches to fixed, predictable paths — low complexity.
4	SSL check	Slightly more involved since it needs certificate inspection rather than just a status code.
5	Broken link checker	Most complex: needs concurrency limits and a link cap. Build it last so the rest of the tool already works while you tune this one.
6	Performance basics	TTFB and page size are cheap; consider swapping in PageSpeed Insights API rather than building your own Lighthouse.

7	Report UI	Build the checklist UI once the API response shape is stable, so you're not re-wiring the frontend every time a check changes.
8	Scoring	A simple weighted score across all ten checks (weight accessibility/SSL higher than nice-to-haves like alt text) — added last since it depends on every other check existing.

4.6 Things to Watch Out For

Timeouts

Every outbound fetch — the main page, robots.txt, sitemap.xml, each link check — needs its own explicit timeout (around 10 seconds is reasonable). Without one, a single slow or unresponsive target site will tie up a request handler indefinitely, and under load this can take your whole service down.

Rate Limiting & Abuse

Because this tool makes outbound requests on behalf of whoever uses it, it can be repurposed as a cheap way to hammer arbitrary third-party sites. Add per-IP rate limiting on your own API, and consider capping how many links per scan and how many scans per URL per hour you'll allow.

Server-Side Request Forgery (SSRF)

Because the backend fetches whatever URL a user submits, a malicious user could submit an internal address (e.g. `http://localhost`, `169.254.169.254` for cloud metadata endpoints, or an internal-network IP) to probe your own infrastructure. Resolve the hostname before fetching and reject private/loopback/link-local IP ranges.

CORS / Bot Blocking

Some sites block automated requests outright (via user-agent checks or firewalls). Handle a blocked or unusual response gracefully — report it as "could not verify" rather than a hard fail, and consider setting a descriptive, honest user-agent string identifying your tool.

Large Sites

Checking every link on a site with thousands of pages synchronously will time out the request. Cap the link count in the MVP, and move this check to a background job in Phase 2 so the user gets the rest of the report immediately and the link results fill in afterward.

Redirects

Decide upfront whether to follow redirects transparently or treat a redirect itself as noteworthy. Either way, always report the final URL you actually scanned, not just the one the user typed in — otherwise a report on `http://example.com` that silently redirected to a totally different domain will be confusing.

4.7 Nice-to-Haves for Later

- **Downloadable PDF report** — export the same JSON report as a shareable document for handing off to a client or stakeholder.
- **Historical comparison** — re-scan a previously checked URL and diff the new report against the old one to show what improved or regressed.
- **Shareable report link** — a public, read-only URL for a given report, useful for sharing with a client without giving them tool access.
- **Scheduled recurring checks** — run the scan automatically on a schedule and alert by email if something that used to pass starts failing.
- **Multi-page crawl** — extend beyond the homepage to check a handful of key pages (e.g. discovered via the sitemap) instead of just the entry point.

End of document.